EPIC-D — Kudos System and Answer Ranking — Technical Spec

Status: Draft — for stakeholder review **Date**: 14 July 2026 **Author**: Adrian Hall (Technical Lead), drafted with Claude Code **Scope**: The fourth per-epic technical spec. Builds on the architecture spec (`docs/superpowers/specs/2026-07-14-askapeer-architecture-design.md`, Section 4.2 for `community.kudos` and the "Redis hot-path caching" note in Section 3) and EPIC-C's forum spec (`docs/superpowers/specs/2026-07-14-epic-c-forum-technical-spec.md`), which this epic's ranking logic sits inside. Also touches EPIC-B's `community.handles.kudos_total` column, which this epic is the sole writer of (per the boundary EPIC-B's spec, Section 9, already declared).

Source of truth: `docs/askapeer-prd-v0.1.md`, the Must-have "Kudos system" row (Section 6.1) and Section 9.2 (kudos as a visible reputation signal).

Small in scope compared to EPIC-C, but load-bearing: kudos-based ranking, not seniority, is the PRD's stated mechanism for "ideas win on merit."

Contents

---

1. Scope

In scope: awarding kudos to a post or comment, un-awarding (Section 3), the write path to `community.handles.kudos_total`, and the ranking logic that orders comments within a thread.

Out of scope: the forum data model itself (EPIC-C), the profile display of `kudos_total` (EPIC-B), moderation actions (EPIC-F) beyond the interaction described in Section 7.

---

2. Data model

Reuses `community.kudos`, already fully defined in the architecture spec, Section 4.2:

```
community.kudos
  id, target_type(post|comment), target_id, given_by_handle_id,
created_at
  unique(target_type, target_id, given_by_handle_id)   -- one
kudos per handle per item
```

No new table is needed. This epic's only schema-level responsibility is the write path into `community.handles.kudos_total` (owned column, EPIC-B spec Section 9) — implemented as a same-transaction increment/decrement alongside the `community.kudos` insert/delete, not a separately-scheduled recomputation. Keeping the write atomic with the triggering action is what makes `kudos_total` trustworthy as a reputation figure — a batch-recompute job would introduce a window where a profile's displayed total lags reality, which matters more here than it would for a less identity-adjacent number, precisely because kudos is one of the few things a peer can see about a handle at all (PRD Section 9.2).

---

3. Awarding and un-awarding kudos

- **One kudos per handle per target**, enforced by the existing unique constraint — a member can't inflate a single answer's score by repeated awards.
- **Self-kudos is rejected**: a handle cannot award kudos to its own post or comment. Not stated in the PRD, but follows directly from the same "ideas win on merit" thesis kudos exists to serve — unrestricted self-kudos would let any member manufacture their own reputation number, which undermines the entire signal `kudos_total` is meant to be.
- **Un-awarding is supported** (`DELETE`, Section 5) — a member can retract a kudos they gave. Not specified in the PRD; proposed here because kudos is presented to members as a live, editable action ("award kudos to contributions you find valuable"), and locking it permanently the moment it's given has no clear justification and would surprise users. Flagged in Section 10 in case there's a reason (e.g. gaming via rapid award/retract cycles to manipulate visible-at-a-glance thread state) to restrict this.
- **Target must be** `published` (EPIC-C's `status` field) — a `removed` post/comment cannot receive new kudos, though kudos already given before removal are not retracted (Section 7).

---

4. Answer ranking

Within a single post's thread, top-level comments (answers) are ordered by kudos count, descending, per PRD Section 6.1 ("Answers within a thread are ranked by kudos, highest first"). Ties are broken by `created_at` ascending — earliest answer wins a tie. This is this spec's own proposal, not a PRD-specified rule, but it's a defensible default: it doesn't reward being first outright (kudos count still dominates), only breaks genuine ties in favour

of whoever contributed the answer earliest, which seems more defensible than an arbitrary/random tiebreak or than favouring the most recent (which would reward answer-sniping a popular thread just before it's viewed).

Nested replies (replies to a comment, via `parent_comment_id`) are ordered chronologically within their parent, not kudos-ranked — the PRD's ranking language is specifically about "answers within a thread," i.e. top-level responses to the original question, not every level of nested discussion. Ranking every nesting level by kudos would fragment genuine back-and-forth conversation threads in a way the PRD's framing doesn't ask for.

Ranking is computed at read time from `community.comments` joined against `community.kudos` counts (or the cached `kudos_total`-equivalent per comment — see Section 6), not stored as a precomputed order column, since kudos changes are relatively low-frequency events per thread compared to the read volume of viewing a thread, and dynamic ordering is simpler to keep correct than a maintained order column.

---

5. API endpoints

```
POST   /v1/posts/:post_id/kudos              -- award kudos to a
post
DELETE /v1/posts/:post_id/kudos               -- retract
POST   /v1/comments/:comment_id/kudos         -- award kudos to a
comment
DELETE /v1/comments/:comment_id/kudos         -- retract
```

Both return the target's updated kudos count. No separate "read kudos count" endpoint is needed — it's already part of the post/comment representation returned by EPIC-C's `GET /v1/posts/:post_id`.

---

6. Leaderboards and the Redis cache

The architecture spec (Section 3) names "kudos leaderboards" as the example of Redis's hot-path caching role. This epic is where that's made concrete: a Redis sorted set (`kudos:leaderboard`, member = `handle_id`, score = `kudos_total`) updated alongside the same write that updates `community.handles.kudos_total` in Postgres. Any "top contributors" view (not itself in the PRD's MVP scope as a named feature, but a natural extension of a reputation system, and useful for the moderation/admin picture of who's active) reads from Redis rather than running a `ORDER BY kudos_total DESC` query against Postgres on every request — the same reasoning the architecture spec gives generally for using Redis as a read-path accelerator rather than a source of truth. Postgres remains authoritative; Redis is a derived, rebuildable cache.

---

7. Interaction with moderation and status

- **Kudos already awarded to content later removed by EPIC-F are not retracted** — `kudos_total` reflects historical contribution, not a live audit of currently-published content. Removing a popular comment doesn't

retroactively strip its author's earned reputation; the two are treated as separate concerns (this spec's judgment call, flagged in Section 10, since the PRD doesn't address it directly).
- **A** `suspended` **or** `expelled` **handle cannot award new kudos** (their session token doesn't authenticate, per the architecture spec's Section 7.2 mechanism) **and cannot receive new kudos** (EPIC-B spec, Section 7, already establishes this) — this epic just needs to reject `POST .../kudos` targeting such a handle's content with the same reasoning already established there, not a new rule.

---

8. Non-functional notes specific to EPIC-D

- **No** `identity`**-schema involvement** — kudos is purely a `community`-schema, handle-to-handle interaction; no new cross-schema regression test surface beyond what EPIC-C already established.
- **Write contention**: a popular post could see many concurrent kudos awards; the unique constraint on `community.kudos` combined with a straightforward `UPDATE ... SET kudos_total = kudos_total + 1` (not a read-then-write) avoids a race condition where two simultaneous awards could otherwise both read the same starting total.

---

9. Test plan

- **Uniqueness**: a handle awarding kudos twice to the same target is rejected (idempotent — the second call either 409s or is treated as a no-op, needs picking, see Section 10).
- **Self-kudos rejection**: awarding kudos to your own post/comment is rejected.
- **Ranking**: comments are returned kudos-descending, `created_at`-ascending on ties; nested replies remain chronological regardless of kudos.
- **Retract**: `DELETE` removes the `community.kudos` row and decrements `kudos_total`; total never goes negative.
- **Removed-content kudos persistence**: removing a comment via EPIC-F's moderation action does not change the kudos-giver's or kudos-receiver's totals.
- **Leaderboard consistency**: the Redis sorted set score matches Postgres's `kudos_total` after a burst of awards/retractions (a consistency check worth running in integration tests, not just unit tests, given it's a cache being kept in sync manually rather than via a single transactional write).

---

10. Open questions

- **Idempotency of a repeated award**: should calling `POST .../kudos` a second time (already-awarded) 409, or silently succeed as a no-op? Minor, but needs picking before the client's error-handling is built.
- **Un-award / retract**: confirmed as this spec's own addition (Section 3) — worth Adrian/Andrew confirming it's actually wanted, versus kudos being a one-way, permanent signal (which some reputation systems

deliberately choose, to prevent rapid award/retract gaming of a visible score).

- **Reputation clawback on moderation**: should a member who receives a formal warning or suspension have kudos from the offending content clawed back, or does `kudos_total` stay purely historical regardless of what happens to the underlying content (Section 7)? No PRD guidance either way.
- **"Top contributors" leaderboard as a member-facing feature**: not in the PRD's MVP scope at all — Section 6's Redis design assumes it might exist eventually (or serves an admin/ops view), but it isn't a committed feature; flagged so it isn't mistaken for confirmed scope.